

Desarrollo Web y Mobile — Resumen de estudio (JavaScript)

Generado a partir de los artículos de MDN abiertos: Closures, Equality comparisons, Strings, Data structures, Language overview, What is JavaScript. Fecha: 11/08/2026.

1. ¿Qué es JavaScript?

- Es la tercera capa de la web: HTML da estructura, CSS da estilo y JavaScript aporta el comportamiento dinámico e interactivo.
- Se ejecuta en un "entorno de ejecución" (normalmente la pestaña del navegador); cada pestaña está aislada por seguridad y no puede acceder directamente al código de otra.
- El código corre en general de arriba hacia abajo, por lo que el orden de declaración importa (aunque el hoisting puede alterar este comportamiento).
- Las APIs del navegador (DOM, Geolocation, Canvas/WebGL, audio/video) y las APIs de terceros (mapas, redes sociales) amplían lo que se puede lograr con JS.
- Es un lenguaje interpretado: el navegador ejecuta el texto fuente directamente, aunque los motores modernos usan compilación "just-in-time" en tiempo de ejecución para acelerar el rendimiento.
- Código "client-side" corre en el navegador del usuario; código "server-side" (PHP, Python, Node.js, etc.) corre en el servidor y devuelve resultados ya procesados.
- Una página es dinámica si puede actualizar su contenido sin recargarse completamente; es estática si siempre muestra lo mismo.
- Formas de incorporar JS: script interno (al final del body), script externo (archivo .js enlazado, idealmente con type="module" o el atributo defer) y handlers inline como onclick, que se consideran mala práctica.
- `addEventListener()` es la forma recomendada de asociar comportamiento a eventos en vez de atributos inline.
- Los comentarios se escriben con doble barra (una línea) o con barra-asterisco (multilínea) y el navegador los ignora.

2. Tipos de datos y estructuras de datos

- JavaScript es de tipado dinámico y débil: las variables no están ligadas a un tipo fijo y el lenguaje convierte tipos automáticamente cuando lo necesita (coerción implícita).
- Tipos primitivos: Number, BigInt, String, Boolean, Symbol, Undefined y Null. Todo lo demás se considera un Object.
- typeof aplicado a null devuelve "object" (particularidad histórica del lenguaje); para comprobarlo hay que comparar con === null.
- undefined indica ausencia de valor (variable no inicializada, propiedad inexistente, función sin return); null indica una ausencia intencional de objeto.
- Number usa el formato de doble precisión IEEE 754; fuera del rango seguro de enteros puede perder precisión. Existen los valores especiales NaN e Infinity; NaN es el único valor que nunca es igual a sí mismo.
- BigInt permite enteros de magnitud arbitraria (se escribe con el sufijo n); no puede mezclarse con Number en operaciones aritméticas sin conversión explícita.
- Los strings son secuencias de texto codificadas en UTF-16 e inmutables: sus métodos siempre devuelven cadenas nuevas, nunca modifican la original.
- Symbol crea valores únicos e inmutables, útiles como claves de propiedades que no colisionan con otras.
- Los objetos son la única estructura mutable; agrupan propiedades clave-valor, que pueden ser "data properties" (value, writable, enumerable, configurable) o "accessor properties" (con funciones get/set).
- Otras estructuras integradas: Array (listas ordenadas), Map y Set (colecciones de asociaciones o valores únicos), WeakMap/WeakSet (similares pero con claves débiles para optimizar memoria), Date y JSON (formato de intercambio de datos basado en texto).
- La coerción de tipos sigue distintos caminos según el contexto (coerción primitiva, numérica o a string), cada uno con un orden distinto de prioridad entre los métodos internos del objeto.

3. Cadenas de texto (Strings)

- Se declaran con comillas simples, dobles o comillas invertidas (template literals); la comilla de apertura y cierre debe coincidir.
- Los template literals permiten insertar expresiones dentro de la cadena y escribir texto multilínea de forma directa, sin necesidad de caracteres especiales de salto de línea.
- La concatenación con el operador + une cadenas; si se mezcla con un número, este se convierte automáticamente a texto.
- Para incluir una comilla dentro de un string se puede usar otro tipo de comilla para delimitar el texto, o escapar el carácter con una barra invertida antes de la comilla.

- `Number()` convierte una cadena en número; `String()` convierte cualquier valor en texto.

4. Comparaciones de igualdad

- JavaScript ofrece tres formas de comparar valores: igualdad débil (`==`, con conversión de tipos), igualdad estricta (`===`, sin conversión) y `Object.is()` (comparación de "mismidad").
- `===` es la opción recomendada en la mayoría de los casos: si los tipos difieren siempre es `false`; con `NaN` nunca da `true` (ni comparado consigo mismo), y `+0 === -0` es `true`.
- `==` convierte los operandos a un tipo común antes de comparar, lo que puede dar resultados poco intuitivos.
- `Object.is()` no realiza conversión de tipos, trata a `NaN` como igual a sí mismo y distingue entre `+0` y `-0` (a diferencia de `===` y `==`).
- La igualdad "same-value-zero", usada internamente por métodos como `includes()`, `Map` y `Set`, se comporta como `===` salvo que trata a `NaN` como igual a `NaN`.
- `null` y `undefined` son iguales entre sí con `==` pero no con `===`.

5. Closures (clausuras)

- Una clausura es la combinación de una función junto con las variables de su entorno léxico en el momento en que fue creada.
- Se generan automáticamente cada vez que se define una función; permiten que una función interna siga accediendo a las variables de su función externa incluso después de que esta haya terminado de ejecutarse.
- El "lexical scoping" determina el acceso a variables según el lugar donde fueron declaradas en el código fuente, no según dónde se invoca la función.
- Antes de ES6 solo existía scope de función y global (con `var`); `let` y `const` introdujeron el scope de bloque.
- Las clausuras permiten emular métodos y variables privadas mediante el patrón de módulo (una función que se ejecuta inmediatamente, o IIFE), asociando datos a funciones de forma parecida a la programación orientada a objetos.
- Error clásico: crear clausuras dentro de un bucle usando `var` provoca que todas comparen la misma variable compartida, quedando con su último valor. Se soluciona usando `let/const` en el bucle, o mediante una función fábrica que genere un entorno léxico nuevo en cada iteración.
- Crear funciones innecesarias dentro de otras funciones tiene un costo de rendimiento; los métodos de un objeto deberían normalmente definirse en el prototype en lugar de recrearse en cada instancia.

6. Panorama general del lenguaje

- Variables: `let` (bloque, reasignable), `const` (bloque, no reasignable aunque sí se puede mutar si es un objeto), `var` (función/global, evitar en código moderno). `let/const` tienen una "zona muerta temporal" antes de su línea de declaración.
- Operadores: aritméticos (+, -, *, /, %, **), de asignación compuesta (+=, -=, etc.), de comparación (menor, mayor, ==, ===, etc.) y lógicos (&&, ||, !), que funcionan según la "truthiness" (verdad/falsedad) de un valor.
- Estructuras de control: `if/else` (no existe `elif` propiamente), `while`, `do...while`, `for`, `for...of` (itera valores de un iterable), `for...in` (itera propiedades enumerables de un objeto), `switch`, y `try/catch/throw` para el manejo de errores.
- Los objetos son colecciones de pares clave-valor (similar a diccionarios o hashmaps); las claves siempre son strings o symbols.
- Los arrays son objetos especiales con la propiedad `length`; conviene mantenerlos "densos", sin huecos entre índices.
- Las funciones son ciudadanas de primera clase: se pueden asignar a variables, pasar como argumentos y devolver desde otras funciones. Existen declaraciones de función, expresiones de función, funciones flecha e IIFE.
- Los parámetros `rest` agrupan los argumentos extra en un array; el operador `spread` expande un array en argumentos individuales; también existen parámetros por defecto y destructuring para simular parámetros nombrados.
- Las clases se instancian con `new`; admiten miembros estáticos (`static`) y campos u métodos privados marcados con el símbolo `#`.
- La programación asíncrona se apoya en un bucle de eventos de un solo hilo; las tres formas idiomáticas de escribirla son callbacks, promesas (`then`) y `async/await` (azúcar sintáctico sobre promesas).
- Los módulos usan las palabras clave `import` y `export`; la forma exacta de resolver rutas de módulos depende del entorno de ejecución (navegador, Node.js, etc.).

Práctica: 30 preguntas de opción múltiple

Elegí una opción por pregunta. Las respuestas correctas están al final del documento.

1. ¿Qué combinación de tecnologías forma la capa de comportamiento dinámico en una página web?

- a) HTML
- b) CSS
- c) JavaScript
- d) JSON

2. ¿Por qué cada pestaña del navegador se considera un entorno de ejecución separado?

- a) Para ahorrar memoria
- b) Por seguridad, para que el código de una pestaña no afecte a otra
- c) Porque lo exige el estándar HTML
- d) Para acelerar la carga de imágenes

3. ¿Cuál es la forma recomendada de asociar una función a un evento de un botón, en vez de usar el atributo onclick?

- a) eval()
- b) addEventListener()
- c) document.write()
- d) setInterval()

4. ¿Por qué se dice que JavaScript es un lenguaje interpretado aunque los motores modernos usen compilación just-in-time?

- a) Porque nunca se compila
- b) Porque la compilación ocurre en tiempo de ejecución, no de forma anticipada
- c) Porque no genera código binario
- d) Porque solo corre en el servidor

5. ¿Cuál es la diferencia principal entre código client-side y server-side?

- a) El client-side corre en el navegador del usuario; el server-side corre en el servidor
- b) No hay diferencia
- c) El server-side siempre es más rápido
- d) El client-side no puede usar variables

6. ¿Qué devuelve el operador typeof al aplicarse sobre null?

- a) "null"
- b) "undefined"
- c) "object"
- d) "number"

7. ¿Cuál de los siguientes NO es un tipo de dato primitivo en JavaScript?

- a) Symbol
- b) BigInt
- c) Array
- d) Boolean

8. ¿Qué valor especial resulta de operaciones matemáticas inválidas, como dividir 0/0?

- a) Infinity
- b) NaN
- c) undefined
- d) null

9. ¿Cómo se declara un valor de tipo BigInt?

- a) Agregando el sufijo n al número (por ejemplo 10n)
- b) Usando comillas
- c) Con la palabra clave big
- d) Con el prefijo 0x

10. ¿Por qué se dice que los strings en JavaScript son inmutables?

- a) Porque no se pueden concatenar
- b) Porque una vez creados no pueden modificarse; los métodos devuelven nuevas cadenas
- c) Porque solo aceptan números
- d) Porque no tienen propiedad length

11. ¿Qué tipo de comillas permite usar template literals con interpolación de expresiones?

- a) Comillas simples
- b) Comillas dobles
- c) Comillas invertidas (backticks)
- d) Ninguna de las anteriores

12. Si se necesita incluir una comilla doble dentro de un string delimitado también por comillas dobles, ¿qué se puede hacer?

- a) Nada, es imposible
- b) Escaparla con una barra invertida o usar otro tipo de comilla para delimitar
- c) Usar solo números
- d) Usar la función Boolean()

13. ¿Qué función convierte una cadena de texto en un número?

- a) String()
- b) Number()
- c) Boolean()
- d) Array()

14. ¿Qué ocurre al concatenar un string con un number usando el operador +?

- a) Lanza un error
- b) El número se convierte automáticamente a string y se concatenan
- c) El string se convierte en 0
- d) Se produce NaN

15. ¿Cuál operador de igualdad realiza conversión de tipos antes de comparar?

- a) ===
- b) ==
- c) Object.is()
- d) typeof

16. Con el operador === ¿cuál es el resultado de comparar NaN === NaN?

- a) true
- b) false
- c) undefined
- d) Lanza error

17. ¿Qué hace especial a Object.is() respecto a === en el caso de los ceros?

- a) Es idéntico en todos los casos
- b) Object.is() distingue entre +0 y -0, mientras que === los trata como iguales
- c) Object.is() siempre devuelve false
- d) === no puede comparar números

18. ¿Qué resultado da la expresión "0" == false ?

- a) true
- b) false
- c) undefined
- d) Lanza un TypeError

19. La igualdad same-value-zero (usada por Array.prototype.includes) se diferencia de === porque...

- a) No permite comparar strings
- b) Trata a NaN como igual a NaN
- c) Convierte todo a booleano
- d) Nunca es simétrica

20. ¿Qué es una clausura (closure) en JavaScript?

- a) Un tipo de bucle
- b) La combinación de una función junto con las variables de su entorno léxico al momento de crearse
- c) Un método para eliminar variables
- d) Una forma de comentar código

21. ¿Qué determina el lexical scoping?

- a) El orden en que se ejecutan las promesas
- b) El acceso a variables según el lugar donde fueron declaradas en el código fuente
- c) El tipo de dato de una variable
- d) El nombre de la función

22. ¿Qué problema clásico ocurre al crear clausuras dentro de un bucle usando var?

- a) El programa no compila
- b) Todas las clausuras comparten la misma variable, quedando con el último valor asignado en el bucle
- c) Se crea una copia distinta de la variable en cada iteración automáticamente
- d) El bucle nunca termina

23. ¿Cuál es una solución común al problema de closures en bucles del punto anterior?

- a) Usar let o const en lugar de var, o usar una función fábrica
- b) Eliminar el bucle
- c) Usar Object.is()
- d) Convertir el bucle en un string

24. ¿Qué patrón permite emular métodos y variables privadas usando closures antes de la existencia de clases?

- a) El patrón de módulo (IIFE con entorno léxico compartido)
- b) JSON.stringify()
- c) El operador spread
- d) Array.prototype.map()

25. ¿Cuál de estas declaraciones de variables NO permite reasignación?

- a) var
- b) let
- c) const
- d) Todas permiten reasignación

26. ¿Qué bucle se utiliza para iterar directamente sobre los valores de un array u otro iterable?

- a) for...in
- b) for...of
- c) while
- d) switch

27. ¿Qué hace el operador rest (...args) en la definición de una función?

- a) Agrupa los argumentos adicionales en un array
- b) Elimina argumentos
- c) Convierte argumentos a string
- d) Sirve solo para clases

28. ¿Cómo se declaran los campos privados dentro de una clase en JavaScript?

- a) Con la palabra clave private
- b) Con un guion bajo
- c) Precediendo el nombre con #
- d) No es posible declarar campos privados

29. ¿Cuáles son las tres formas idiomáticas de escribir código asíncrono en JavaScript?

- a) if/else, for, while
- b) Callbacks, promesas y async/await
- c) Arrays, objetos y funciones
- d) Map, Set y WeakMap

30. ¿Qué palabras clave se usan para exportar e importar funcionalidad entre módulos de JavaScript?

- a) include / require
- b) load / save
- c) export / import
- d) module / package

Respuestas correctas

1c · 2b · 3b · 4b · 5a · 6c · 7c · 8b · 9a · 10b · 11c · 12b · 13b · 14b · 15b · 16b · 17b · 18a ·
19b · 20b · 21b · 22b · 23a · 24a · 25c · 26b · 27a · 28c · 29b · 30c